

Outfit Generator

By: Lewhat Kahsay, Isaiah Moliga-Puleyasi, and Joselyn Quinteros

Project Overview: The Outfit Generator is designed to help users choose their outfits by using a system that generates personalized outfit recommendations based on wardrobe data, real-time weather conditions, and user preferences. The system evaluates clothing compatibility using style, color, and texture rules and ranks outfit combinations using a weighted scoring algorithm. It also tracks clothing wear history, separates clean and dirty items, and provides wardrobe statistics such as most and least worn items.

GitHub Repository: <https://github.com/isaiahmoligapuleyasi/62finalproject.git>

Needfinding Results

The initial idea for this project came from a common and relatable daily issue that we all encounter: choosing what to wear. Our group noticed that this was an issue we and our roommates struggled with nearly everyday, often spending an unnecessary amount of time deciding on outfits or repeating the same outfit. Thus, our original idea was a simple digital closet that could randomly suggest outfits from stored clothing items.

We conducted informal interviews with our roommates, since we see them every day and often observe them struggling with the same problem. During one interview, a friend emphasized that weather conditions heavily influence what she wears and she checks the weather app every morning to decide on an outfit. Another person mentioned that she cares about outfit style coordination and appearance. These conversations helped us understand that our outfit selection needs a combination of practicality, weather awareness, and personal style preferences. Moreover, while talking among ourselves, we realized that laundry separation would be a helpful feature to implement because people cannot wear items that are dirty so it would not be practical to include those items in the outfit generator.

There are also problems that are not well-suited for software solutions in regard to generating an outfit. Hence, fashion is subjective and influenced by several factors such as trends, emotions, and personal identity, all which cannot be fully captured by rules or algorithms. Decisions like feeling confident or expressing identity through clothing are human and cannot be entirely modeled through computation alone. Therefore, our software is designed to assist decision-making and offer the best possible outfit options based on the input provided.

Dataset

The dataset used for this project is stored as comma-separated values (a CSV), where each row represents a single clothing item. This structured format was chosen because it is simple to parse in Java and easy to modify manually if needed. We created the dataset by writing in 20 clothing items and then asking Claude to generate the rest. Each item includes the following attributes: name, color, texture, style, category, and temperature suitability. We categorized items into meaningful

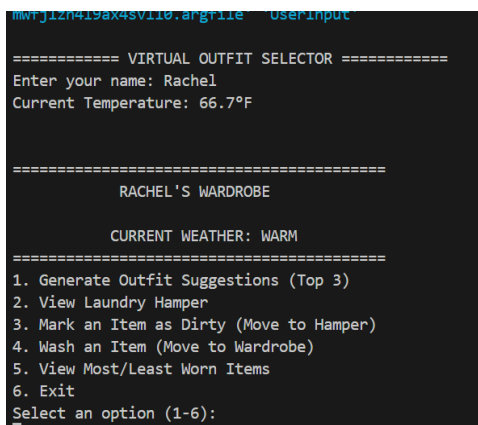
groups such as tops, bottoms, jackets, and dresses to support outfit generation logic. The temperature attribute was included later on in the timeline to integrate weather-based recommendation.

Data Structure

The main data structure we used in this project is a set of `ArrayList<ClothingItem>` collections inside the `Wardrobe` class. Each clothing category which includes tops, bottoms, jackets, dresses, and the laundry hamper has its own separate list. This design illustrates how people organize clothing in real life by category. It also makes the system easier to navigate and understand when building outfits. Instead of searching through one large collection of clothing items, the program is able to access the relevant categories. Thus, we choose `ArrayList` because it supports dynamic resizing, which is important for a wardrobe that changes. Hence, clothing items are being moved between lists when they are marked as dirty or clean. This approach balances simplicity and sufficient performance for the project's needs.

Results (Features Demonstration)

This project demonstrates several key features through console output in the main program. The program first prompts the user to input their name ("Rachel"). After the user enters their name, the program displays a menu-driven interface where users have the option to do several things and the current temperature in Claremont is displayed under the user's name. The user can then choose to generate outfit suggestions, view laundry hampers, wash or make any items dirty, or track clothing statistics.



```
nmrj12n419ax4sv110.argfile - UserInput
===== VIRTUAL OUTFIT SELECTOR =====
Enter your name: Rachel
Current Temperature: 66.7°F

=====
RACHEL'S WARDROBE

CURRENT WEATHER: WARM
=====
1. Generate Outfit Suggestions (Top 3)
2. View Laundry Hamper
3. Mark an Item as Dirty (Move to Hamper)
4. Wash an Item (Move to Wardrobe)
5. View Most/Least Worn Items
6. Exit
Select an option (1-6):
```

The primary feature of our application is outfit generation. Below is an example of a user selecting option 1 from the menu. As one can observe, the outfit generation feature displays the top three ranked outfit combinations based on compatibility scoring from the users responses. The user can

choose to wear one of the suggested options, which increases the wear count on each clothing item included in that outfit. Alternatively, the user may decide they do not like any of the options and skip

the selection process which will bring them back to the main menu.

```

CURRENT WEATHER: WARM
=====
1. Generate Outfit Suggestions (Top 3)
2. View Laundry Hamper
3. Mark an Item as Dirty (Move to Hamper)
4. Wash an Item (Move to Wardrobe)
5. View Most/Least Worn Items
6. Exit
Select an option (1-6):
1

===== OUTFIT PREFERENCES=====

What type of outfit would you like?
[A] Tops/Bottoms
[B] Jackets/Dresses
Enter: A

What is the style of the outfit?
- Casual
- Formal
- Athletic
- Loungewear
- Outerwear
Enter style: Casual

Is there a specific color you want to wear?
(or type 'any'): black

Would you like a jacket? (yes/no)
yes

--- TOP 3 RANKED SUGGESTIONS ---
1: Long sleeve (Black) + Overalls (Black) + Fleece vest (Black/White)
2: Long sleeve (Black) + Palazzo pants (Black) + Fleece vest (Black/White)
3: Long sleeve (Black) + Pleated pants (Black) + Fleece vest (Black/White)

Pick an outfit (1-3) or 0 to skip: 
```

Option number 2 allows the user to view the laundry hamper which is where dirty items are stored when the user marks an item as dirty. However, there are no items currently in there since we have not marked any items as dirty. Thus, before we select option 2, we will mark an item as dirty as demonstrated below.

```

RACHEL'S WARDROBE

CURRENT WEATHER: WARM
=====
1. Generate Outfit Suggestions (Top 3)
2. View Laundry Hamper
3. Mark an Item as Dirty (Move to Hamper)
4. Wash an Item (Move to Wardrobe)
5. View Most/Least Worn Items
6. Exit
Select an option (1-6):
3

--- MARK ITEM AS DIRTY ---
Enter part of the item name (ex: 'halter', 'jeans', 'raincoat'): Long sleeve

Multiple items found. Please choose:
1: Long sleeve (Burgundy)
2: Long sleeve (Black)
3: Long sleeve (Plaid Blue/Green)
4: Long sleeve (Floral Multicolor)
5: Thermal long sleeve (Mauve)
6: Long sleeve (Cobalt)
Enter the number of the item: 2
Found it! 'Long sleeve' moved to laundry.
```

When a user selects option 3 they are prompted to enter part of the clothing item's name or the full name. If there are multiple matches to the name then all options will show and a user chooses which one is the item they wish to mark as dirty. From here we can now check the laundry hamper. We see that our dirty item has been placed in the hamper.

```
=====
RACHEL'S WARDROBE
=====
CURRENT WEATHER: WARM
=====
1. Generate Outfit Suggestions (Top 3)
2. View Laundry Hamper
3. Mark an Item as Dirty (Move to Hamper)
4. Wash an Item (Move to Wardrobe)
5. View Most/Least Worn Items
6. Exit
Select an option (1-6):
2

--- LAUNDRY HAMPER ---
- Long sleeve (Black)
```

Option 4 allows the user to wash clothing items that are currently marked as dirty. When the user selects this option, the program prompts them to enter the full name or part of the name of the clothing item they wish to wash. In the example shown below, the user selects option 4 and enters the word “Long Sleeve” as the item they wish to wash. Once selected, the clothing item is removed from the laundry hamper and marked as clean again.

```

RACHEL'S WARDROBE

CURRENT WEATHER: WARM
=====
1. Generate Outfit Suggestions (Top 3)
2. View Laundry Hamper
3. Mark an Item as Dirty (Move to Hamper)
4. Wash an Item (Move to Wardrobe)
5. View Most/Least Worn Items
6. Exit
Select an option (1-6):
4
Enter the name of the item you washed: Long Sleeve
'Long sleeve' is now clean and back in the wardrobe.

=====
RACHEL'S WARDROBE

CURRENT WEATHER: WARM
=====
1. Generate Outfit Suggestions (Top 3)
2. View Laundry Hamper
3. Mark an Item as Dirty (Move to Hamper)
4. Wash an Item (Move to Wardrobe)
5. View Most/Least Worn Items
6. Exit
Select an option (1-6):
2

--- LAUNDRY HAMPER ---
The hamper is empty.

=====

```

Option 5 allows the user to track clothing usage statistics by viewing the most and least worn items across different clothing categories. These categories include tops, bottoms, jackets, and dresses. The program analyzes the wear count associated with each clothing item and identifies which items have been worn the most frequently and which have been worn the least.

This feature helps users monitor usage and clothing rotation over time. Additionally, when a clothing item is washed using option 4, its wear count resets back to zero. This simulates how users would track how many times an item has been worn since its last wash. Whereas the least worn are items that the user could consider wearing since it is clean.

```

=====
RACHEL'S WARDROBE

CURRENT WEATHER: WARM
=====
1. Generate Outfit Suggestions (Top 3)
2. View Laundry Hamper
3. Mark an Item as Dirty (Move to Hamper)
4. Wash an Item (Move to Wardrobe)
5. View Most/Least Worn Items
6. Exit
Select an option (1-6):
5

--- MOST/LEAST WORN STATISTICS ---
Most Worn Top: Long sleeve (2 wears)
Least Worn Top: tank (0 wears)
-----
Most Worn Bottom: Cargo pants (1 wears)
Least Worn Bottom: shorts (0 wears)
-----
Most Worn Jacket: Cropped cardigan (1 wears)
Least Worn Jacket: Winter puffer (0 wears)
-----
Most Worn Dress: Midi dress (1 wears)
Least Worn Dress: Casual dress (0 wears)

```

Option 6 ends the program.

```

===== VIRTUAL OUTFIT SELECTOR =====
Enter your name: Rachel
Current Temperature: 73.6°F

=====
RACHEL'S WARDROBE

CURRENT WEATHER: WARM
=====
1. Generate Outfit Suggestions (Top 3)
2. View Laundry Hamper
3. Mark an Item as Dirty (Move to Hamper)
4. Wash an Item (Move to Wardrobe)
5. View Most/Least Worn Items
6. Exit
Select an option (1-6):
6
Goodbye!

```

Analysis “Section”

Feature 1: Outfit Recommendation based on texture, color, style, and weather

Class/Methods : *UserInput Class (and most, if not at all, methods involved in prompting outfit generation)*

Pre-condition and edge cases: Outfit generating largely depends on user input and ensuring that all the methods involved work properly; outfit generating is largely dependent on methods rooted in other classes (and many edge cases are handled with exception/print statements to redirect users). For example, one precondition involves the user prompting the outfit generator with various clothing choices (i.e. style): casual, formal, athletic, loungewear, outerwear. While there's a print statement (an exception) to prevent invalid entries, there's intuition here that users will enter correct input (else the program would be unable to score the style/and overall outfit properly). Another precondition for outfit generation, is that it's important to ensure that the wardrobe has at least **one item** in each of the category lists (list1 and list2), in the case that either are empty, allPossibilities stays empty (an edge case), prompting “No items found” to print for users.

Time complexity: The time complexity of outfit generating is **$O(T*B*J)$** where T = top, B = bottom, and J = jacket; or, $O(T*B)$ in the case a user doesn't select a jacket). Essentially what our outfit generates is looking for a top to then pair with a bottom to then a jacket (if a user chooses). Since outfit generating is not just one method (many methods involved: this is an overall runtime with respect to what outfit generation is actually doing). Most methods involved in outfit generating are a series of constant-time operations (i.e. isColorCompatible or isTextureCompatible). Collections.sort runs an $O(T*B*(\log T*B))$, but this is a slower growth than the overall run time, $O(T*B*J)$, so that dominates.

Space complexity: The space complexity for outfit generating is largely influenced by the allPossibilities array list which stores all the OutfitOption for all the various top-bottom pairs ($T*B$ entries), thus **$O(T*B)$** space complexity. All other methods involved in the outfit generating process, like scoring variables or string comparisons are constant space (they're just single local variables that

are being overwritten as the code goes).

Feature 2: Laundry hamper that keeps track of dirty clothing items

Class/Methods: `markItemDirty` and `markItemClean`

Preconditions/edge cases: For `markItemDirty` (moving a clothing item to the laundry hamper), the clothing item must currently exist in the list of clean clothing items (and also you must prompt `markItemDirty` with some clothing item). So, if you call `markItemDirty` on a dirty clothing item, the `remove()` call will not work and the dirty clothing item gets added to the laundry hamper (again); this creates a duplicate of dirty clothing items in the laundry hamper. Originally, one edge case revolved around `completeMarkDirty` which utilizes a `String.contains()` to search for clothing items. Nonetheless, we noticed that prompting it with simple string names (i.e. “shirt” could result point to: “t-shirt”, “dress shirt”, “sweatshirt”, and other shirt-related items). To fix this there is a list that’s printed to ask users what item they want to clean. For `markItemClean`, it requires the same precondition to `markItemDirty` (prompting a clothing item to it). It differs however because in `completeItemWash`, we do not display a list of possible clean items, it will just find the first match that a user inputs and clean that. For the most part this shouldn’t be a big issue, nonetheless one possible edge case arises if items share a similar name.

Time complexity: The time complexity for this feature is **$O(n)$** as the methods involved only run a fixed number of linear scans through category lists, where n is just the total number of clothing items they scan through. `markItemDirty` calls `remove()` on four lists, `markItemClean` calls `laundryHamper.remove()` then `addClothingItem()` which scans category keywords, these are all linear operations. None of these are nested either so all these operations stay $O(n)$.

Space complexity: The space complexity for this feature is **$O(n)$** . `completeMarkDirty` builds a matches `ArrayList` which in the worst case holds all n clean clothing items (if every item matches the inputted search string). `completeItemWash` calls `getlaundryHamper()` which returns a copy of the hamper, also up to n items.

Feature 3: Least-worn and most-worn counts to identify least/most worn pieces

Class/Methods: displayStats, getMostWornTop, getLeastWornTop, getMostWornBottom, getLeastWornBottom, getMostWornJacket, getLeastWornJacket, getMostWornDress, getLeastWornDress in Wardrobe

Preconditions/edge cases: Each method checks for null/empty lists and returns null, which displayStats handles by printing out a no clothing item to be found statement, so empty categories are safe. Only **clean** items are scanned. A heavily worn item sitting in the hamper won't show up in stats, which can give a misleading picture of your actual most worn piece. If all items in a category have the same wear count, both most and least worn return the first element in the list, technically correct but not very informative. resetWearCount() is called on wash, so counts only reflect the clothing item's current clean cycle, not total wear count.

Time complexity: The time complexity for this feature is **O(n)** as they are just single linear scans through category lists. displayStats calls all six sequentially (still just O(n)), where n is total clothing items.

Space complexity: The space complexity of this feature (and these methods) is **O(1)** as each method only ever holds two references (mostWorn, leastWorn) regardless of how many items exist. No lists are built or copied, so it's constant space.

Feature 4: Live weather integration using an external weather API

Class/Methods: getCurrentTemperature in WeatherService.java

Preconditions/edge cases: Largely, this feature requires an active internet connection and the Open-Meteo API to be available. On any failure it silently defaults to "warm", so the user gets no indication their outfits may be weather-inappropriate (given that we base our weather API in Pomona/Claremont, this "warm" weather seems likely). Coordinates are hardcoded to Claremont/Pomona so the app always reports that location's weather regardless of where the user

actually is. A possible edge case could be if a user is not locally located in Claremont/Pomona, opportunity for incorrect outfit generation. `getCurrentTemperature` is called once at startup and reused the entire session, so the weather may be inaccurate if the program is not restarted/reprompted. Another edge case is that the weather string returned must exactly match the temperature strings stored on `ClothingItem` objects, any casing mismatch would give a zero for scoring during weather check in outfit generation.

Time complexity: The time complexity for this feature is **$O(1)$** because it is a single HTTP request followed by a fixed amount of JSON parsing with no loops or data structures that might impact overall run time.

Space complexity: The space complexity is **$O(1)$** because only a fixed set of local variables are allocated: the `HttpClient`, `HttpRequest`, response string, and `JsonObject`. None of these change (space wise) throughout the program.

Affordance Selection

In our project, we believe that college-students, but really *anyone* who enjoys/or finds use in assembling outfits would benefit from most of our software projects. Keeping this in mind, we believe that in future iterations, our project could extend to ensure that clothing items could keep close attention to inclusive body-types and abilities. For example, perhaps wheelchair-users could store information about clothing they find works best based on their mobility and our software could work to accommodate that persons' needs more adequately. Additionally, this may raise a need to add possible physical accessories or ability-supporting tools to our software so that people can consider how mobility-devices/ability-supporting tools might coincide with their outfit (or, just be a part of the outfit itself).

When thinking about how we're comparing different clothing item's information to pick a best fit outfit, it comes a lot out of our own interests and beliefs as programmers: for example we picked color coordination largely based on our own interests in colors. However, this does not extend to encompass the variety of opinions/views on what people think might work well together. One

thing we believe is powerful about our software however is that it is largely accessible to users who can go in and make edits to the program to create comparisons that best reflect their own interests/personal style.

It's possible that our software could be malicious in the sense that people take it to be a concrete truth. While our program aims to support people in choosing an outfit that best suits them, we're not forcing, nor saying it is the *best* option: a simple algorithm is doing so. At the end of the day, we'd like people to still have the autonomy to choose the outfits that make sense for them, even if it doesn't align with our program's recommendation 100%. Nonetheless, we see it possible that people could twist our recommendations to be concrete truths in the sense of punishing people for not following clothing recommendations or criticizing people who deviate from their assigned recommendations.

A Reflection Session

Between the three programmers (Lewhat, Isaiah, and Joselyn) who worked on this project for over two weeks, we'd say it took us around **~32+ hours** all together to complete it—a long time, and lots of hard work. I think it took us a lot longer than we anticipated (yet we'd probably say we expected it to take a long time nonetheless). The process of creating the project itself was really nice: the team met 2-3 times a week outside of class and did check-ins nearly every class after the final was assigned and we knew we'd be working together. We all took initiative to drive different aspects of the project, each contributing immensely meaningful features that each one of us built on top of, asked questions about, and worked on collaboratively to develop strong features. While many of our core ideas remained (some morphed into different/flushed out ways). For example, we intended for our project to be chatbot style, where users would enter in their idea for an outfit and we'd parse it. However we thought that it made more sense to pivot and do more of a menu/user input (pick a choice) style to ensure that we could program something in the timeframe we had that reflected the outfit generating idea we wanted to complete. Feedback on the final project: in the future, it might be good to have more solidified milestones, after the first check-in it feels like we're left to develop a

lot of the program (in a way that felt a tad bit overwhelming). Overall though, we really enjoyed this working project, we learned so much. While it was difficult and tedious, it was immensely helpful and gratifying to be able to develop a project that culminated all of our learning through CS62 (and we're really proud of our project).